



Politechnika Krakowska **im. Tadeusza Kościuszki**

Wstęp do Cyberbezpieczeństwa

Laboratorium 4

Nr albumu: D/161212

Jakub Kozieński

Grupa: CY2

Implementacja algorytmu SHA-256

Stałe algorytmu

H_INIT to 8 wartości początkowych stanu skrótu, pochodzących z części ułamkowych pierwiastków kwadratowych z pierwszych 8 liczb pierwszych.

Tablica K zawiera 64 stałe rundowe, wyprowadzone z części ułamkowych pierwiastków sześciennych z pierwszych 64 liczb pierwszych.

MASK32 służy do przycinania wyników operacji do 32 bitów, bo Python nie posiada natywnych typów o stałej szerokości.

```
19 # H0..H7
20 # 8 liczb pierwszych (2, 3, 5, 7, 11, 13, 17, 19)
21 H_INIT = [
22     0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
23     0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19,
24 ]
25
26 # K[0..63]
27 # pierwszych 64 liczb pierwszych (2..311)
28 K = [
29     0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
30     0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
31     0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
32     0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
33     0x27b70a85, 0x2e1b2138, 0x4d2c6dfe, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
34     0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
35     0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
36     0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2,
37 ]
38
39 MASK32 = 0xFFFFFFFF # maska do obciecia wyniku do 32 bitow
40
```

Operacje bitowe na słowach 32-bitowych

Rotr realizuje cykliczne przesunięcie bitów w prawo (rotację), gdzie bity "wypadające" z prawej strony wracają od lewej.

Shr to zwykłe przesunięcie logiczne w prawo, gdzie bity wypadające są tracone, a od lewej wpisywane są zera.

Obie funkcje są kluczowymi operacjami pierwotnymi używanymi wewnątrz pętli kompresji.

```
43
44 def rotr(x: int, n: int) -> int:
45     """Rotacja w prawo o n bitow (w ramach 32 bitow)."""
46     x &= MASK32
47     return ((x >> n) | (x << (32 - n))) & MASK32
48
49
50 def shr(x: int, n: int) -> int:
51     """Przesunięcie w prawo o n bitow (z wpisywaniem zer)."""
52     return (x & MASK32) >> n
53
54
```

Padding

Doprowadzenie wiadomości do długości będącej wielokrotnością 512 bitów (= 64 bajtów).

Najpierw dopisywany jest bit 1 (bajt 0x80), potem uzupełnienie zerami aż długość wiadomości modulo 64 wynosi 56, a na końcu dopisywana jest oryginalna długość jako 8-bajtowa liczba big-endian .

```
56
57 def preprocess(message: bytes) -> bytes:
58     original_bit_len = len(message) * 8
59
60     padded = message + b'\x80'
61
62     while (len(padded) % 64) != 56:
63         padded += b'\x00'
64
65     # 64-bitowa dlugosc oryginalnej wiadomosci w bitach, big-endian
66     padded += original_bit_len.to_bytes(8, byteorder='big')
67     return padded
68
```

Przygotowanie harmonogramu wiadomości

Jest to wykonywane dla każdego 512-bitowego bloku osobno, zaraz przed wejściem w pętlę kompresji.

Pierwsze 16 słów ($w[0..15]$) to prostu surowe dane z bloku, podzielone na 32-bitowe słowa.

Pozostałe 48 słów ($w[16..63]$) jest rozszerzanych za pomocą wzoru mieszającego poprzednie wartości, co zapewnia dobre rozprzestrzenienie się danych wejściowych po całym harmonogramie.

```
70 def build_message_schedule(block: bytes) -> list:
71     """
72     Tworzenie tablicy w[0..63] dla jednego 512-bitowego bloku.
73     """
74     w = [0] * 64
75     # 16 slow po 32 bity = 64 bajty bloku
76     for i in range(16):
77         w[i] = int.from_bytes(block[i * 4:(i + 1) * 4], byteorder='big')
78
79     for i in range(16, 64):
80         s0 = rotr(w[i - 15], 7) ^ rotr(w[i - 15], 18) ^ shr(w[i - 15], 3)
81         s1 = rotr(w[i - 2], 17) ^ rotr(w[i - 2], 19) ^ shr(w[i - 2], 10)
82         w[i] = (w[i - 16] + s0 + w[i - 7] + s1) & MASK32
83     return w
84
```

Pętla kompresji jednego bloku

S0 i S1 to funkcje sigma, które mieszają bity przez kombinację rotacji i przesunięć, co wprowadza nieliniowość.

```
86 def compress(h: list, w: list) -> list:
87     """
88     Pętla kompresji dla jednego bloku.
89     """
90     a, b, c, d, e, f, g, hh = h
91
92     for i in range(64):
93         S1 = rotr(e, 6) ^ rotr(e, 11) ^ rotr(e, 25)
94         ch = (e & f) ^ (~e & MASK32 & g)
95         temp1 = (hh + S1 + ch + K[i] + w[i]) & MASK32
96
97         S0 = rotr(a, 2) ^ rotr(a, 13) ^ rotr(a, 22)
98         maj = (a & b) ^ (a & c) ^ (b & c)
99         temp2 = (S0 + maj) & MASK32
100
101         hh = g
102         g = f
103         f = e
104         e = (d + temp1) & MASK32
105         d = c
106         c = b
107         b = a
108         a = (temp1 + temp2) & MASK32
109
110     return [
111         (h[0] + a) & MASK32,
112         (h[1] + b) & MASK32,
113         (h[2] + c) & MASK32,
114         (h[3] + d) & MASK32,
115         (h[4] + e) & MASK32,
116         (h[5] + f) & MASK32,
117         (h[6] + g) & MASK32,
118         (h[7] + hh) & MASK32,
119     ]
120
```

Główna funkcja realizująca algorytm haszowania SHA-256.

Wykonuje padding, dzieli dane na 512-bitowe bloki i dla każdego bloku przygotowuje harmonogram wiadomości oraz pętlę kompresji. Zwraca 256-bitowy skrót w postaci 64-znakowego ciągu szesnastkowego.

```
122 def sha256(message) -> str:
123     if isinstance(message, str):
124         message = message.encode('utf-8')
125
126     padded = preprocess(message)
127     h = list(H_INIT)
128
129     # kazdy blok ma 512 bitow = 64 bajty
130     for offset in range(0, len(padded), 64):
131         block = padded[offset:offset + 64]
132         w = build_message_schedule(block)
133         h = compress(h, w)
134
135     return ''.join(f'{value:08x}' for value in h)
136
137
```

Funkcja obliczająca dystans hamminga

```
139
140 def hamming_distance_hex(hex_a: str, hex_b: str) -> int:
141     """Dystans Hamminga między dwoma hashami zapisanymi jako hex - liczony na bitach."""
142     a = int(hex_a, 16)
143     b = int(hex_b, 16)
144     return (a ^ b).bit_count()
145
```

Porównanie czasu wykonania mojej implementacji z funkcją biblioteczną

Różnice w czasie wykonania pojedynczego skrótu są znaczące, ale wynika to w głównej mierze z faktu, że biblioteka hashlib korzysta z implementacji w języku C, które jest szybszy od Pythona.

Porównanie czasu generowania pojedynczego skrótu:			
rozmiar wejścia	moja [ms]	hashlib [ms]	stosunek
11 B (hello world)	0.1133	0.0004	290x
64 B (1 blok)	0.2305	0.0004	567x
1 KB	1.9829	0.0008	2534x
10 KB	18.8549	0.0045	4214x
100 KB	183.3750	0.0399	4594x

Czy można skrócić tekst „dowolnej” długości?

Przetestowano działanie funkcji SHA-256 dla wiadomości o długościach: 1 B, 4 B, 16 B, 64 B oraz 128 B. Dla każdego z testowanych rozmiarów wejścia algorytm poprawnie wytworzył skrót o stałej długości 256 bitów

Test długości wejścia:			
zadane bity	realne bajty	dl. skrotu [bit]	skrot
1	1	256	8a8950f7623663222542c9469c73be3c4c81bbdf019e2c577590a61f2ce9a157 [OK]
32	4	256	a97d8324a3c4a972cf2a9a6a1a828e0be618811bce2929d92291273e0557fe6c [OK]
128	16	256	7ed6a4c6e830e583ddf8c92199e9336444f2a723284598d720844828ad0eed07 [OK]
512	64	256	2c5b1516002de803f041b8c1b362794b231ad329248d96c739d0c02b3a49c82f [OK]
1024	128	256	eea80c393ae45369837e7a753b369742873dfc6bd67e7cf5401af38c8c46835a6 [OK]

Czy po zmianie 1 losowego bitu, skrót z tego ciągu „istotnie” różni się od skrótu z wyjściowej wiadomości?

Tak, skrót po nawet drobnej zmianie wiadomości różni się istotnie od skrótu oryginalnego. Dla pary 'hello world' vs 'hello world!' odległość Hamminga między skrótami wyniosła 131 / 256 bitów $\approx 51,2\%$. Oznacza to, że ponad połowa wszystkich bitów wyniku uległa zmianie, mimo że wiadomości różnią się tylko jednym znakiem na końcu.

m1	m2	Hamming / 256
'hello world'	'hello world!'	131

Czy biorąc dwa różne teksty i skracając je, ich odległość Hamminga jest „duża” ?

Dla 256-bitowego hasha oczekiwana odległość Hamminga między dwoma niezależnymi skrótami wynosi 50 %, czyli ok. 128bitów. Eksperyment, pokazany na rzucie ekranu, na 25 parach różnych tekstów dał średnią 127,48 / 256.

m1	m2	Hamming / 256
'hello world'	'hello world!'	131
'hello world'	'Hello world'	135
'abc'	'abd'	122
'The quick brown fox jumps over the lazy dog.'	'The quick brown fox jumps over the lazy dog.'	136
'test123'	'test124'	116
'password'	'Password'	148
'aaaaaaaa'	'aaaaaaab'	128
'00000000000000000000'	'00000000000000000001'	106
'Ala ma kota'	'Ala ma kotA'	124
'lorem ipsum'	'lorem ipsuM'	125
'2025-01-01'	'2025-01-02'	135
'user@example.com'	'User@Example.co'	122
'wKc Lab 4'	'WkC Lab 4'	128
'kryptografia'	'KRYPTOGRAFIA'	126
'SHA-256'	'Sha-255'	143
'Jarek Marek'	'jAREK darek'	127
'politechnika'	'POLITECHNIKA'	127
'0'	'1'	125
''	','	142
'A'	'B'	124
'klucz-tajny-001'	'klucz-tajny-002'	127
'xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx' + "XXXXXXXXXXXXXX"	"XXX" + "XXXXXXXXXXXXXX"	138
'test'	'Test'	115
'abc123'	'Abc124'	126
'dane wejsciowe'	'Dane wejscioWe'	119

Srednia: 127.48 / 256 (oczekiwane ~128 dla dobrego hasha)

Czy tworzony jest skrót o zawsze tej samej długości ?

Tak, dla dowolnej długości 1 B czy 128 B zawsze tworzony jest skrót o długości 256 bitów.

Test dlugosci wejścia:				
zadane bity	realne bajty	dl. skrotu [bit]	skrot	
1	1	256	8a8950f7623663222542c9469c73be3c4c81bbdf019e2c577590a61f2ce9a157	[OK]
32	4	256	a97d8324a3c4a972cf2a9a6a1a8280be618811bce2929d92291273e0557fe6c	[OK]
128	16	256	7ed6a4c6e830e583ddf8c92199e9336444f2a723284598d720844828ad0eed07	[OK]
512	64	256	2c5b1516002de803f041b8c1b362794b231ad329248d96c739d0c02f3ba49c82f	[OK]
1024	128	256	eea80c393ae45369837e7a753b369742873dfcbbd67e7cf5401af38c8c46835a6	[OK]